

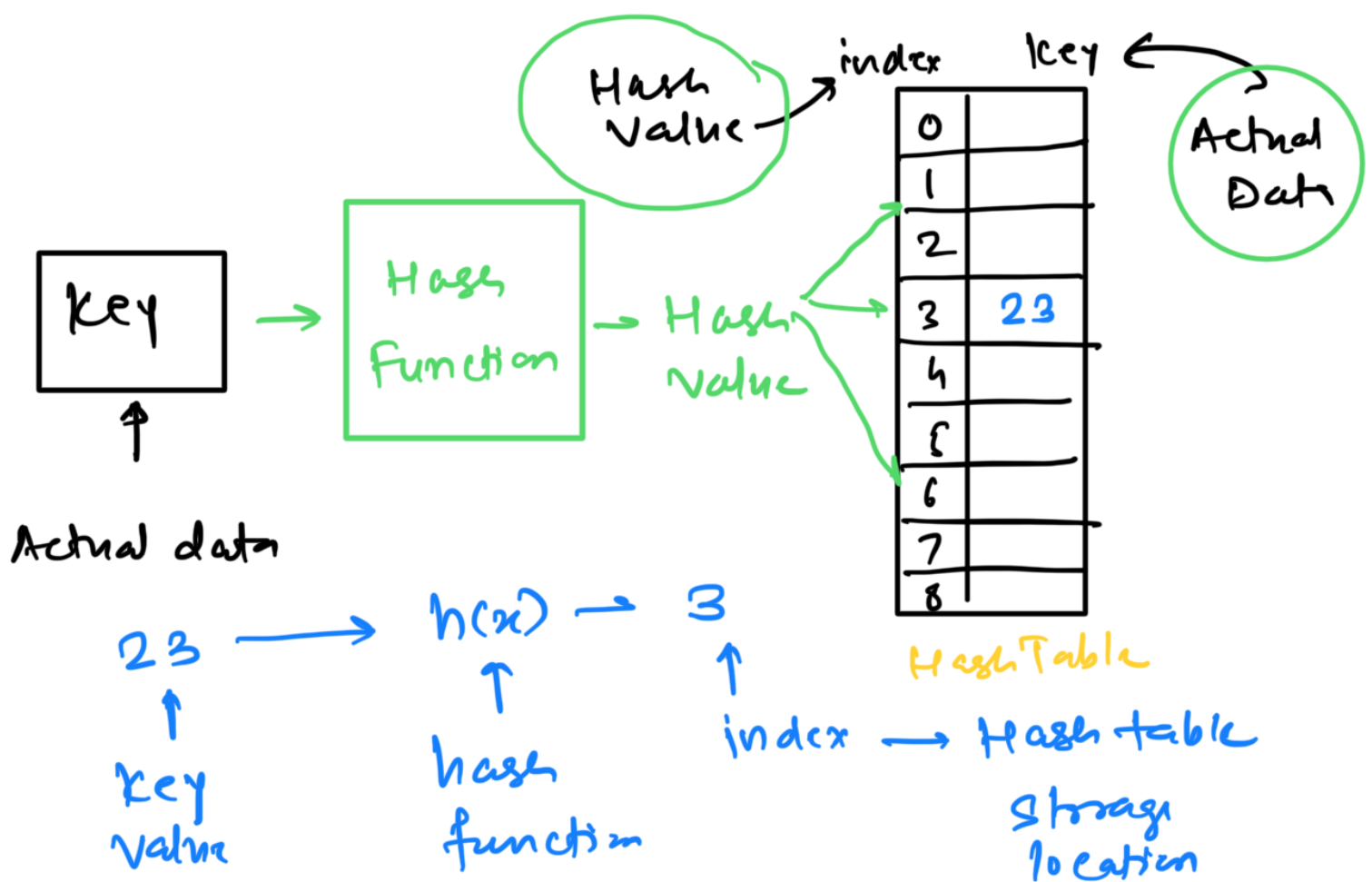
Hashing → fastest search → hashing

→ HashMap → background → Hash Table
↑
Hashing

Def: A technique that determines an index or location for storage of an element in a data structure.

— Hash function: $h(x)$

fastest search → Return's index



Hash Table - data structure that stores elements and allows insertion, lookups (search) and deletion to be performed in $O(1)$ time

- Hash Table is an alternative method for representing a dictionary.

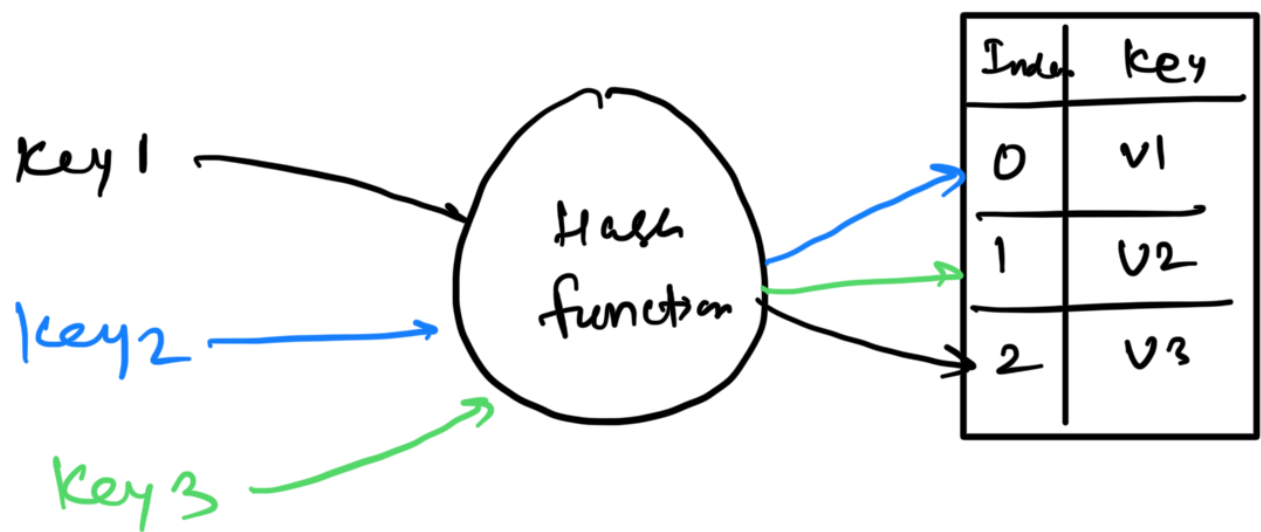
Hash function - It is used to map keys

into position in the table, this is

called as hashing

Operations -

1. Insertion : Compute hash index - $O(1)$
2. Deletion : Compute hash index - $O(1)$
3. Search : Compute hash index - $O(1)$



Ex: key: 8, 3, 13, 6, 4, 10, 50

$$h(x) = \text{key} \% (\text{size})$$

$$h(x) = \text{key} \% (\text{Bucket size})$$

$$h(x) = \text{key} \% 10$$

$$8 \% 10 = 8$$

$$3 \% 10 = 3$$

$$13 \% 10 = 3$$

$$6 \% 10 = 6$$

$$4 \% 10 = 4$$

$$50 \% 10 = 0$$

$$10 \% 10 = 0$$

Collision

Collision

A diagram of a hash table with 10 buckets, indexed 0 to 9. The buckets contain the following values: 0: 50, 1: 30, 2: (empty), 3: 3, 4: 4, 5: (empty), 6: 6, 7: (empty), 8: 8, 9: (empty). Red circles around 50 and 30 are labeled 'Collision'. Blue circles around 3 and 13 are labeled 'Collision'. Yellow lines connect the calculations for 3 and 13 to bucket 3, and for 50 and 30 to bucket 0.

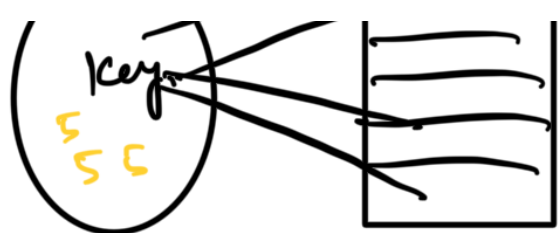
Index	Value
0	50
1	30
2	
3	3
4	4
5	
6	6
7	
8	8
9	

Bucket size = 10

Hash Table size

one to one ✓

one to many ✓



many to many

Load factor in Hashing - (α)

- measure that indicates how full a hash table

-

$$\text{Load factor} = \frac{n}{m}$$

n = Number of elements in the Hash Table

m = Total number of available slots

- It is a ratio of number of elements (n) to the number of buckets (slots) ^(m) in the hash table

Types of Hash Table

1. Open Address Hash Table -

Closed Hashing

Array

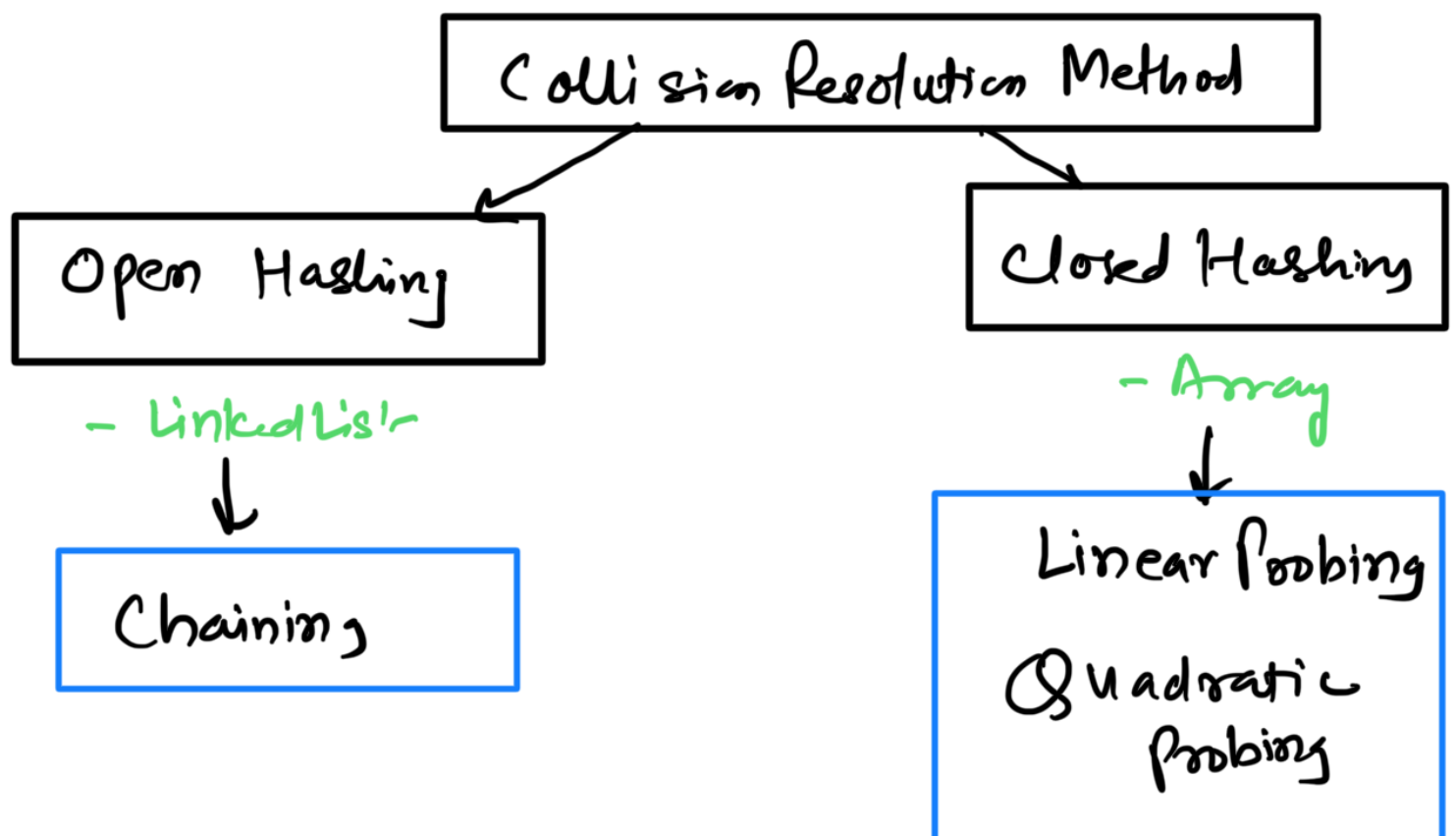
- 1D array indexed by integer values that are computed by an index function called as hash function

Open Hashing

2. Separate Chained Hash Table:

Linked list

- 1D array of linked list indexed by integer values that are computed by an index function called as hash function



Common Hashing Techniques

1. Direct Hashing -

- Each key is mapped directly to a specific index in the hash table

Ex: size = 10, key = 42

$$42 \% 10 = 2$$

index = 2

2. Division Method / Modulo Method

- The key is divided by the table size, and the remainder is used as the index

- Formula: $h(x) = x \% \text{table-size}$

Ex: table-size = 10, key = 42, 23, 73

$$\text{key}1 = 42$$

$$23$$

$$42 \% 10 = 2$$

$$23 \% 10 = 3$$

$$73 \% 10 = 3$$

}
Collision occurs

3. Multiplicative Hashing

- Key is multiplied by a constant, & the fractional part of the result is multiplied by the table size to get the index.

- Formula -

$$h(x) = \text{floor}(\text{table size} * (\text{key} * A \% 1))$$

where $A = \text{Constant } (0 \& 1)$

4. Folding Method

- The key divided into equal parts and the parts are added to get the hash index

- Ex: 987 | 654

$$987 + 654 = 1641$$

The index of $(1641 \% \text{table-size})$

5) Mid-Square Method

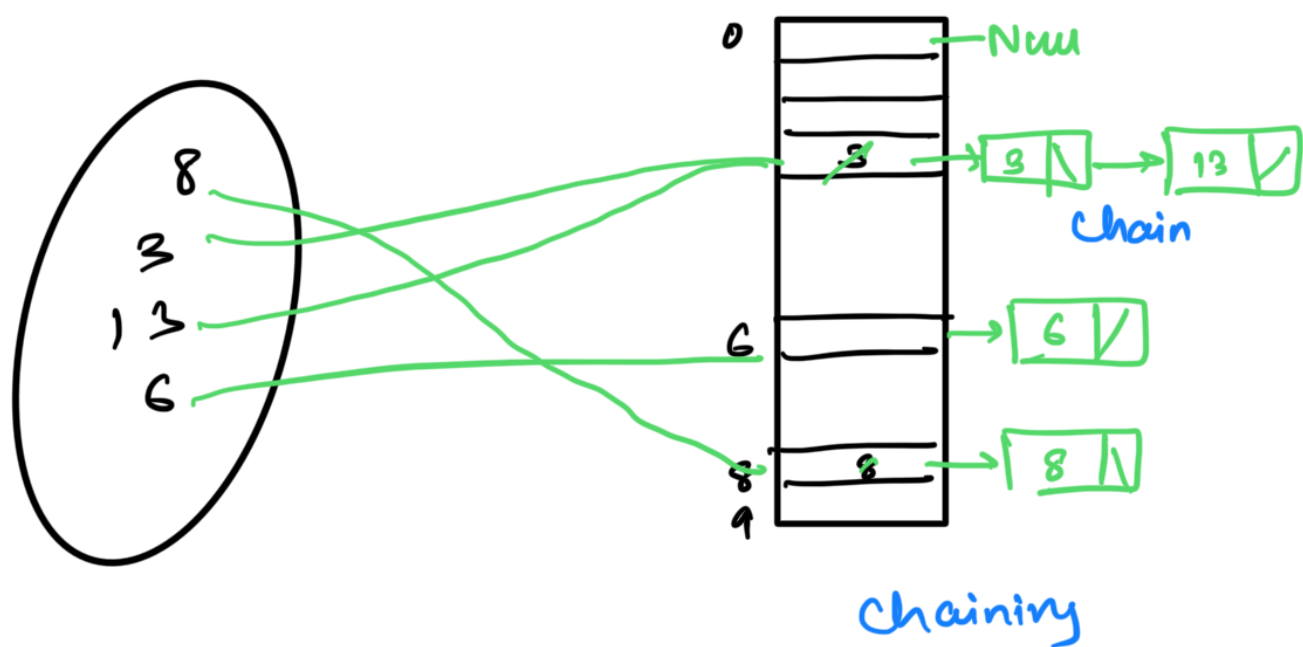
- Key is squared, and the middle digits are extracted from the index

Ex: Key = 4567

$$(4567)^2 = 20857489$$

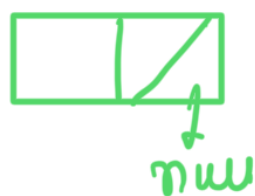
574 % tabb - size

Chaining : LinkedList Collision Handling $\rightarrow$ LL



$$h(x) = x \% \text{size}$$

$\uparrow$
10



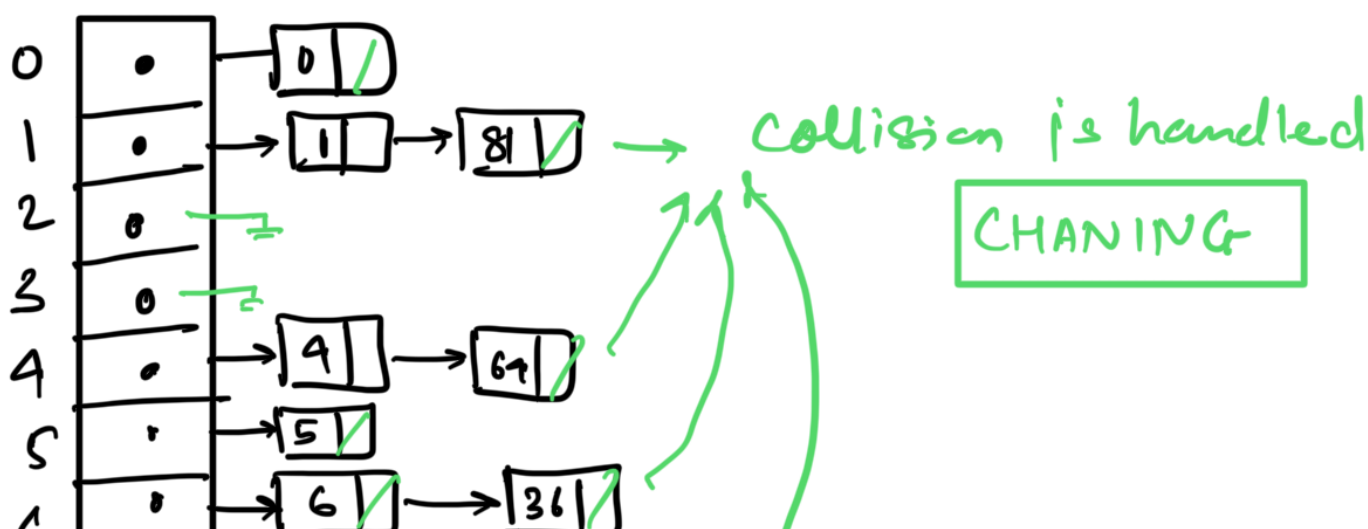
Each slot in the hash table points to a linked list (or chain) of entries to hash to the same index.

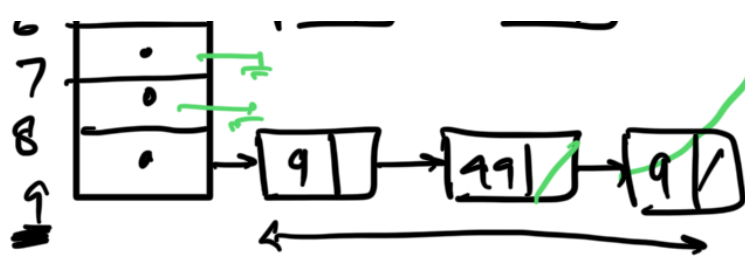
Adv - Handle multiple collisions

Ex: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81

$$\text{hash}(\text{key}) = \text{key} \% 10$$

Handling Collision using chaining



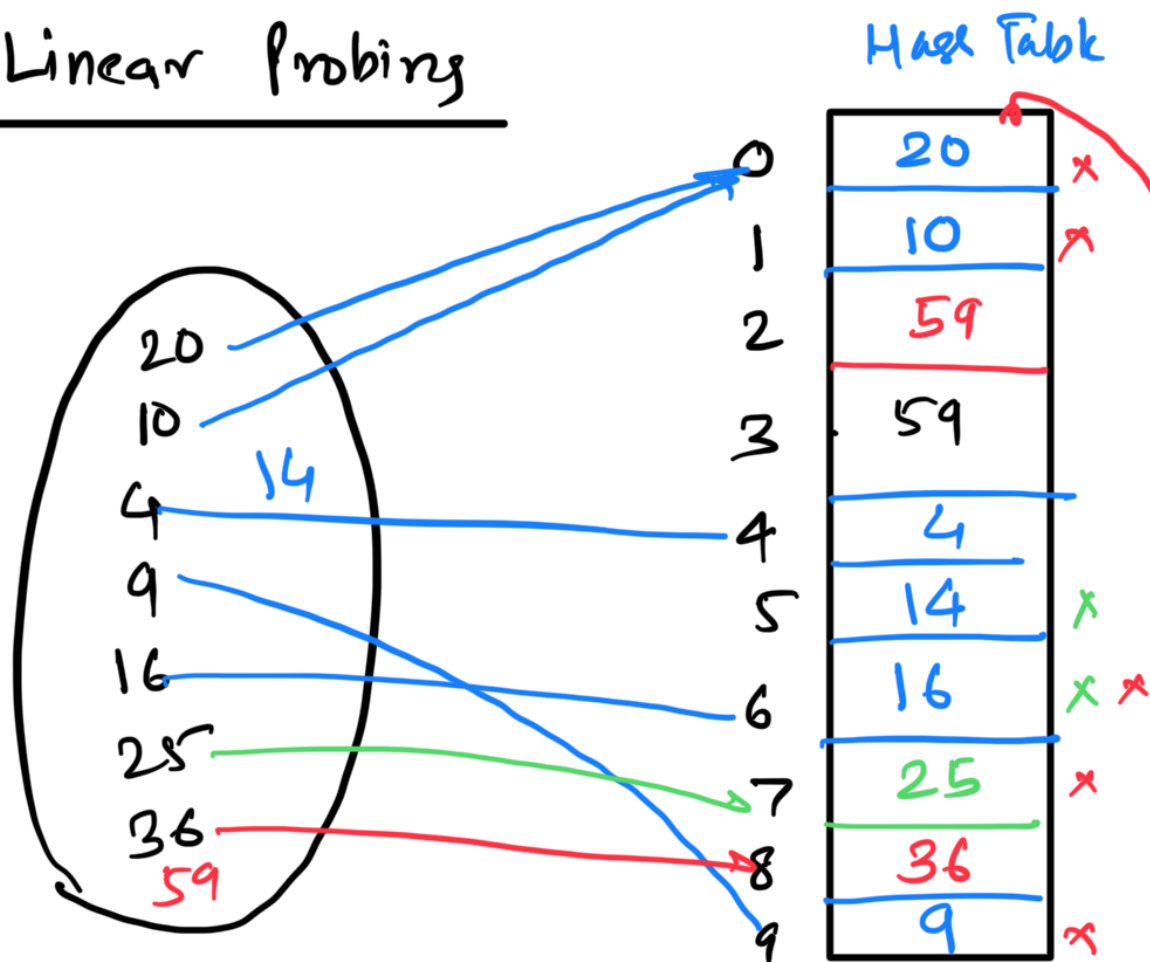


Index → Many

Open Addressing → Arrays : Linear probing

— Collisions are resolved by finding another empty slot within the hash table

— Linear Probing



$$h(x) = x \% 10 \\ = 20 \% 10 = 0$$

LINEAR PROBING

Collision Handling Techniques → LL → Chaining
Arrays — Linear Probing

Open Hashing

Linked List

Separate Chaining

Closed Hashing

Array

Open Address

Chaining
Dynamic

Linear probing
Fixed

Hash Table Time Complexity

	Average	Worst Case
Insert :	$O(1)$	$O(n)$
Delete :	$O(1)$	$O(n)$
Search :	$O(1)$	$O(n)$
Space :	$O(n)$	$O(n)$